perfectly possible to mix minimal APIs and web API controllers in the same application, I would favor consistency over using the new hotness.

Question 5 considers how familiar you already are with web API controllers. If you're coming from legacy ASP.NET or have already used web API controllers in ASP.NET Core and need to be productive quickly, you might decide to stick with web API controllers.

I consider this one of the weaker arguments, as minimal APIs are conceptually simpler than web API controllers; if you already know web API controllers, you will likely pick up minimal APIs easily. That said, the differences in the model binding approaches can be a little confusing, and you may decide it's not worth the investment or frustration if things don't work as you expect.

The final question comes down entirely to taste and preference: do you like minimal APIs? Web API controllers heavily follow the "convention over configuration" paradigm (though not to the extent of MVC controllers and Razor Pages). By contrast, you must be far more explicit with minimal APIs. Minimal APIs also don't enforce any particular grouping, unlike web API controllers, which all follow the "action methods in a controller class" pattern.

Different people prefer different approaches. Web API controllers mean less manual wiring up of components, but this necessarily means more magic and more rigidity around how you structure your applications.

By contrast, minimal API endpoints must be explicitly added to the `WebApplication` instance, but this also means you have more flexibility around how to group your endpoints. You can put all your endpoints in Program.cs, create natural groupings for them in separate classes, or create a file per endpoint or any pattern you choose.